Лабораторная работа №2

Симметричная криптография. СТБ 34.101.31-2011

Пояснительная записка к защите. Реализация: internal/crypto/belt, язык Go

---

Содержание

1. Что сделано в работе 2
2. Обозначения 2
3. Паспорт алгоритма 3
4. Устройство шифра 3
   - 4.1. Подстановка H и функция G 3
   - 4.2. Такт зашифрования 4
   - 4.3. Такт расшифрования 4
   - 4.4. Ключевое расписание 4
   - 4.5. Один такт целиком 5
   - 4.6. Перестановка регистров в коде 5
5. Режимы работы 5
   - 5.1. Простая замена, `belt-ecb` (п. 6.2) 5
   - 5.2. Гаммирование с обратной связью, `belt-cfb` (п. 6.4) 6
   - 5.3. Сводная таблица двух режимов 7
6. Реализация 7
   - 6.1. Структура проекта 7
   - 6.2. Формат шифртекста 8
   - 6.3. Проверки 8
   - 6.4. Порядок байтов 8
7. Проверка работоспособности 8
8. Вопросы на защите 9

1. Что сделано в работе

Реализован блочный шифр *belt* из стандарта СТБ 34.101.31-2011 и два режима его применения: режим простой замены (п. 6.2) и режим гаммирования с обратной связью (п. 6.4). Это ровно то, что требуется заданием; остальные режимы стандарта (сцепление блоков, счётчик, имитовставка, хеширование) не делались и в коде отсутствуют.

Пункт	Что	Функции Go	Контрольные примеры
п. 6.1	Блочное преобразование F, F^{-1}	`BlockEncrypt` / `BlockDecrypt`	A.1, A.4
п. 6.2	`belt-ecb` – простая замена	`ECBEncrypt` / `ECBDecrypt`	A.9-1, A.9-2
п. 6.4	`belt-cfb` – гаммирование с обратной связью	`GammaWithFeedbackEncrypt` / `GammaWithFeedbackDecrypt`	A.13

Все результаты совпали побитово с примерами приложения А. Круговые проверки (зашифровал – расшифровал, получил то же самое) пройдены отдельно.

> **Ключевые числа для защиты**
> Блок – **128 бит**. Ключ – **256 бит**. Тактов – **8**, по 7 тактовых ключей на такт, всего 56 обращений к ключу. Подстановка – **одна таблица на 256 байт**. Режимы: простая замена и гаммирование с обратной связью.

2. Обозначения

Запись	Смысл
СТБ	Государственный стандарт Республики Беларусь. Полное наименование: СТБ 34.101.31-2011 «Информационные технологии и безопасность. Алгоритмы шифрования и контроля целостности». Редакция 2020 года перенесла алгоритмы без изменений.
belt	Название блочного шифра стандарта (Belarusian Standard). В тексте стандарта режимы именуются `belt-ecb`, `belt-cfb` и т. д.
X, Y	Открытый текст и шифртекст. $X = X_1 \| \ldots \| X_n$, где все блоки, кроме последнего, ровно 128 бит.
θ	Ключ, 256 бит. Восемь 32-битных слов: $\theta = (\theta_1, \ldots, \theta_8)$.
S	Синхропосылка, 128 бит. Не секретная, передаётся вместе с шифртекстом.
$F(\cdot)$	Зашифрование одного блока на ключе θ: $F = F^\theta$.
$F^{-1}(\cdot)$	Расшифрование одного блока.
$H(u)$	Подстановка: 8 бит в 8 бит, таблица из 256 байт. К 32-битному слову применяется побайтово.
$G_{r(u)}$	$\mathrm{ROL}_r(H(u))$ – подстановка H и циклический сдвиг слова влево на r бит. Используются $r = 5, 13, 21$.
$u \oplus v$	Побитовое исключающее ИЛИ (XOR).
$u \boxplus v$, $u \boxminus v$	Сложение и вычитание 32-битных слов по модулю 2^{32}.
$L_{s(u)}$	Первые s бит блока u.
$\langle i \rangle_{32}$	Число i, записанное 32-битным словом (номер такта).

> **Ловушка: L_s и порядок байтов**
> Стандарт нумерует байты с начала строки, а в Go блок собирается как little-endian, где первый байт строки – **младший** байт числа. Поэтому «взять L_s» – это `cipherText[:s]`, то есть `s` байт с начала среза, а не маска по старшим битам. Перепутать направление – самая частая ошибка при реализации режимов, и круговые тесты её не ловят, а примеры из приложения А ловят сразу.

3. Паспорт алгоритма

Параметр	Значение
Год, статус	2011, действующий стандарт Республики Беларусь
Тип	симметричный блочный шифр
Структура	SP-сеть на четырёх 32-битных словах, схема типа Лэя–Мэсси. Не сеть Фейстеля.
Размер блока	**128 бит** (16 байт)
Размер ключа	**256 бит** (32 байта). Стандарт допускает также 128 и 192 бита, но в работе всегда 256.
Число тактов	8, по 7 тактовых ключей на такт
Нелинейность	одна таблица H: подстановка 8 бит в 8 бит
Ключевое расписание	отсутствует как преобразование: слова ключа берутся циклически
Порядок байтов	little-endian во всех словах

4. Устройство шифра

4.1. Подстановка H и функция G

Единственный нелинейный элемент – таблица H из 256 байт. Если слово u состоит из байтов u_0, u_1, u_2, u_3 (начиная с младшего), то

$$H(u) = H(u_0) + 2^8 H(u_1) + 2^{16} H(u_2) + 2^{24} H(u_3)$$

а функция G_r сдвигает результат влево на r бит:

$$G_r(u) = \mathrm{ROL}_r(H(u))$$

Разные величины сдвига (5, 13, 21) нужны для того, чтобы биты расходились по слову неодинаково.

```
func hWord(u uint32) uint32 {
  return uint32(hTable[byte(u)]) |
    uint32(hTable[byte(u>>8)])<<8 |
    uint32(hTable[byte(u>>16)])<<16 |
    uint32(hTable[byte(u>>24)])<<24
}

func g(u uint32, r int) uint32 {
  return bits.RotateLeft32(hWord(u), r)
}
```

Таблица хранится в коде шестнадцатеричной строкой и один раз разворачивается в массив при старте пакета (`mustDecodeH`): ошибиться в 512 символах при написании руками легко, а проверяется это только тестами.

4.2. Такт зашифрования

Блок X разбит на четыре 32-битных слова a, b, c, d (слово a – первые четыре байта). Такт с номером i (от 1 до 8) состоит из девяти операций над словами и заканчивается перестановкой регистров:

1. $b \leftarrow b \oplus G_5(a \boxplus K_{7i-6})$
2. $c \leftarrow c \oplus G_{21}(d \boxplus K_{7i-5})$
3. $a \leftarrow a \boxminus G_{13}(b \boxplus K_{7i-4})$
4. $e \leftarrow G_{21}(b \boxplus c \boxplus K_{7i-3}) \oplus \langle i \rangle_{32}$
5. $b \leftarrow b \boxplus e, \quad c \leftarrow c \boxminus e$
6. $d \leftarrow d \boxplus G_{13}(c \boxplus K_{7i-2})$
7. $b \leftarrow b \oplus G_{21}(a \boxplus K_{7i-1})$
8. $c \leftarrow c \oplus G_5(d \boxplus K_{7i})$
9. $a \leftrightarrow b, \quad c \leftrightarrow d, \quad b \leftrightarrow c$

Номер такта $\langle i \rangle_{32}$ на шаге 4 – **раундовая константа**: она делает такты неодинаковыми даже при одном и том же ключе, и без неё шифр разрушается.

После восьмого такта блок собирается в порядке b, d, a, c:

$$Y = b \parallel d \parallel a \parallel c$$

> **Перестановок ровно восемь, а не девять**
> Перестановка из пункта 9 входит в состав такта, поэтому выполняется восемь раз. Последние восемь применений: четыре раза порядок возвращается к $abcd$, значит восемь применений дают тот же порядок, что и два, то есть $bdac$. Формула $Y = b \parallel d \parallel a \parallel c$ не является дополнительной перестановкой: это указание, в каком порядке склеить регистры, уже стоящие в этом порядке.

4.3. Такт расшифрования

Расшифрование идёт тактами $i = 8, 7, \ldots, 1$, причём тактовые ключи внутри такта берутся в обратном порядке, а перестановка регистров другая:

1. $b \leftarrow b \oplus G_5(a \boxplus K_{7i})$
2. $c \leftarrow c \oplus G_{21}(d \boxplus K_{7i-1})$
3. $a \leftarrow a \boxminus G_{13}(b \boxplus K_{7i-2})$
4. $e \leftarrow G_{21}(b \boxplus c \boxplus K_{7i-3}) \oplus \langle i \rangle_{32}$
5. $b \leftarrow b \boxplus e, \quad c \leftarrow c \boxminus e$
6. $d \leftarrow d \boxplus G_{13}(c \boxplus K_{7i-4})$
7. $b \leftarrow b \oplus G_{21}(a \boxplus K_{7i-5})$
8. $c \leftarrow c \oplus G_5(d \boxplus K_{7i-6})$
9. $a \leftrightarrow b, \quad c \leftrightarrow d, \quad a \leftrightarrow d$

$$X = c \parallel a \parallel d \parallel b$$

> **Шаги 4--6 при расшифровании не меняются**
> Операция $(b, c) \to (b + e, c - e)$ не обращается сама в себя в обратном порядке, но шаги 7–9 разворачивают зависимость, так что весь такт обратим. Именно эта асимметрия (одинаковые шаги 4–6, разные 7–9 и разные перестановки) сначала сбивает с толку при реализации.

4.4. Ключевое расписание

Ключ делится на восемь слов младшим байтом вперёд. Преобразований над ними нет, индекс берётся по модулю восьми:

$$K_j = \theta_{(j-1) \bmod 8} + 1$$

За 8 тактов по 7 ключей получается 56 обращений, то есть каждое слово ключа используется семь раз. Никакого расписания в привычном смысле у *belt* нет – это и плюс (быстро), и минус (нет диффузии ключа между тактами).

```
func subkeysForRound(theta [8]uint32, i int, encrypt bool) [7]uint32 {
  var subkeys [7]uint32
  for j := range subkeys {
    if encrypt {
      subkeys[j] = theta[(7*i-7+j)%8]
    } else {
      subkeys[j] = theta[(7*i-1-j)%8]
    }
  }

  return subkeys
}
```

4.5. Один такт целиком

Операции такта одинаковы для обоих направлений, поэтому в коде это одна функция:

```
func beltRound(a, b, c, d uint32, i uint32, k [7]uint32) (uint32, uint32, uint32, uint32) {
  b ^= g(a+k[0], 5)
  c ^= g(d+k[1], 21)
  a -= g(b+k[2], 13)

  e := g(b+c+k[3], 21) ^ i

  b += e
  c -= e
  d += g(c+k[4], 13)
  b ^= g(a+k[5], 21)
  c ^= g(d+k[6], 5)

  return a, b, c, d
}
```

Сложение и вычитание в Go для `uint32` уже идут по модулю 2^{32}, отдельного кода для этого не нужно.

4.6. Перестановка регистров в коде

Перестановка из пункта 9 такта не выполняется отдельным вызовом: первые семь заданы тем, что следующий такт читает регистры уже в переставленном порядке. Восьмая выполняется явно, потому что следующего такта уже нет.

```
// abcd -> bdac -> dcba -> cadb, повторяется
a, b, c, d = beltRound(a, b, c, d, 1, subkeysForRound(theta, 1, true))
b, d, a, c = beltRound(b, d, a, c, 2, subkeysForRound(theta, 2, true))
d, c, b, a = beltRound(d, c, b, a, 3, subkeysForRound(theta, 3, true))
c, a, d, b = beltRound(c, a, d, b, 4, subkeysForRound(theta, 4, true))
a, b, c, d = beltRound(a, b, c, d, 5, subkeysForRound(theta, 5, true))
b, d, a, c = beltRound(b, d, a, c, 6, subkeysForRound(theta, 6, true))
d, c, b, a = beltRound(d, c, b, a, 7, subkeysForRound(theta, 7, true))
c, a, d, b = beltRound(c, a, d, b, 8, subkeysForRound(theta, 8, true))
```

Блок собирается из четырёх слов little-endian, как и в стандарте.

5. Режимы работы

5.1. Простая замена, `belt-ecb` (п. 6.2)

Если длина сообщения кратна 16 байтам, режим тривиален:

$$Y_i = F(X_i), \quad X_i = F^{-1}(Y_i), \quad i = 1, \ldots, n$$

Если последний блок неполный (r байт, $0 < r < 16$), применяется **кража шифртекста**: недостающие байты берутся у шифртекста предпоследнего блока.

$$\begin{aligned} Y_i &= F(X_i), \quad i = 1, \ldots, n-2 \\ Y^* &= F(X_{n-1}) \\ Y_n &= L_r(Y^*) \\ Y_{n-1} &= F(X_n \parallel (\mathrm{Right})_{16-r}(Y^*)) \end{aligned}$$

То есть предпоследний блок шифруется, его первые r байт уезжают в конец шифртекста, а его хвост склеивается с неполным блоком открытого текста и шифруется второй раз. Расшифрование выполняется той же схемой с F^{-1}.

> **Три шага, которые все путают**
> 1. Блок $X_n \parallel (\mathrm{Right})_{16-r}(Y^*)$ **тоже шифруется** – это второй вызов F, а не просто склейка.
> 2. Длина шифртекста равна длине открытого текста, ничего не дополняется.
> 3. Нужен хотя бы один полный блок: красть байты не у чего, если блок всего один и неполный.

В коде это ровно две функции:

```
lastCipher := BlockEncrypt(data[(n-1)*BlockSize:n*BlockSize], key)

augmented := make([]byte, BlockSize)
copy(augmented, data[n*BlockSize:])
copy(augmented[rem:], lastCipher[rem:])

copy(result[(n-1)*BlockSize:n*BlockSize], BlockEncrypt(augmented, key))
copy(result[n*BlockSize:], lastCipher[:rem])
```

Свойства. Длина сохраняется: $|Y| = |X|$. Синхропосылки нет, шифрование детерминировано, одинаковые блоки открытого текста дают одинаковые блоки шифртекста. Зашифрование и расшифрование параллелятся, есть произвольный доступ к блокам. Ошибка в одном бите шифртекста портит весь соответствующий блок открытого текста.

5.2. Гаммирование с обратной связью, `belt-cfb` (п. 6.4)

Регистр обратной связи – предыдущий блок **шифртекста**:

$$\begin{aligned} & Y_0 = S \\ \text{зашифрование:} \quad & Y_i = X_i \oplus L_{|X_i|}(F(Y_{i-1})) \\ \text{расшифрование:} \quad & X_i = Y_i \oplus L_{|X_i|}(F(Y_{i-1})) \end{aligned}$$

Разница между направлениями ровно одна: что уходит в регистр обратной связи. При зашифровании – результат сложения, при расшифровании – принятый блок шифртекста. В обоих случаях вызывается только F, то есть **обратная функция не нужна вообще**.

```
for offset := 0; offset < len(data); offset += BlockSize {
  end := min(offset+BlockSize, len(data))

  yCurr := BlockEncrypt(yPrev, key)
  for i := offset; i < end; i++ {
    cipherText[i] = data[i] ^ yCurr[i-offset]
  }

  yPrev = cipherText[offset:end]
}
```

Неполный блок всегда последний, поэтому $L_{|X_n|}$ просто отрезает лишнее: никакого дополнения и никакой отдельной обработки хвоста не требуется. Синхропосылка вырабатывается `crypto/rand` при каждом зашифровании.

Свойства. Длина сохраняется, дополнения нет, F^{-1} не используется. Применим к сообщениям любой длины, включая один байт. Зашифрование строго последовательное, расшифрование распараллеливается. Ошибка в бите Y_i портит тот же бит X_i **и весь блок X_{i+1}**, после чего поток восстанавливается – это самосинхронизация через один блок.

5.3. Сводная таблица двух режимов

Свойство	ECB (п. 6.2)	CFB (п. 6.4)
Нужна синхропосылка	нет	да, передаётся открыто
Дополнение	не нужно (кража шифртекста)	не нужно
Минимальная длина	16 байт	1 байт
Длина шифртекста	$\|X\|$	$16 + \|X\|$ с синхропосылкой
Используется F^{-1}	да	нет
Одинаковые блоки X	дают одинаковые Y	дают разные Y
Зашифрование параллелится	да	нет
Расшифрование параллелится	да	да
Ошибка в бите Y_i портит	весь X_i	бит X_i и весь X_{i+1}
Самосинхронизация	нет	да
Что защищает	только секретность	только секретность

> **Про ошибку в последних блоках ECB**
> Из-за кражи шифртекста искажение в Y_{n-1} портит и X_{n-1} целиком, и хвост X_n целиком, потому что этот хвост берётся из расшифровки Y_{n-1}. Ошибка же в самом коротком блоке Y_n портит только сам короткий хвост X_n. Это удобно знать, но на стойкость не влияет.

6. Реализация

6.1. Структура проекта

Файл	Содержимое
`internal/crypto/belt/belt.go`	таблица H, функция G_r, такт, расписание тактовых ключей, `BlockEncrypt` и `BlockDecrypt`
`internal/crypto/belt/ecb.go`	режим простой замены, включая кражу шифртекста для неполного последнего блока
`internal/crypto/belt/gammaFeedback.go`	режим гаммирования с обратной связью
`internal/crypto/belt/belt_test.go`	проверки на примерах приложения А и круговые прогоны
`internal/cipher/belt.go`	обёртка: приём ключа, выбор режима, вывод в виде hex-строки
`main.go`	запуск: выбор шифра, режима, направления, ввод ключа и данных

Пакет `internal/crypto/belt` ничего не знает про пользователя и ничего не знает про вывод: это чистые функции над `[]byte`. Ошибки возвращаются явно (`([]byte, error)`), поэтому проверки длины ключа и длины сообщения живут там же, где алгоритм.

6.2. Формат шифртекста

Синхропосылка в стандарте – внешний параметр протокола. В программе принято соглашение: при зашифровании она вырабатывается случайно и передаётся в начале шифртекста.

$$\text{шифртекст CFB} = \underbrace{S}_{16\text{ байт}} \;\|\; \underbrace{\text{тело}}_{|X|\text{ байта}}$$

Ввод и вывод – шестнадцатеричные строки, поэтому 16 байт синхропосылки занимают первые 32 символа вывода. У простой замены синхропосылки нет, и её вывод имеет ровно такую же длину, как ввод.

6.3. Проверки

Где	Проверка	Почему
`BlockEncrypt`, `BlockDecrypt`	нет	работают с блоком и ключом фиксированной длины, вызываются только изнутри пакета
`validateKey`	$\lvert K\rvert = 32$ байта	ключ стандарта – 256 бит
`ECBEncrypt`, `ECBDecrypt`	$\lvert X\rvert \geq 16$ байт	для кражи шифртекста нужен предыдущий блок
`GammaWithFeedbackEncrypt`	$\lvert X\rvert > 0$	пустое сообщение шифровать нечего
`GammaWithFeedbackDecrypt`	$\lvert Y\rvert > 0, \lvert S\rvert = 16$	синхропосылка передаётся отдельно и должна быть блоком

> **Известный недочёт**
> В обёртке `internal/cipher/belt.go` ошибки ядра отбрасываются: `res, _ := belt.ECBEncrypt(in, key)`. На правильном пути это безвредно, но если пользователь введёт текст короче 16 байт для простой замены, программа молча покажет пустой результат вместо сообщения об ошибке. Исправляется тривиально – пробросить `err` вместо `_`.

6.4. Порядок байтов

Все слова little-endian, блок и слово ключа собираются через `binary.LittleEndian`. Нех-ключ из стандарта вводится байт за байтом, как есть.

7. Проверка работоспособности

Тесты запускаются командой `go test ./internal/crypto/belt/`. Всего 12 тестов, все проходят.

1. **Контрольные примеры стандарта.** Шифрование блока (A.1) и расшифрование блока (A.4) совпали побитово. Простая замена проверена на 48 байтах (A.9-1) и на 47 байтах с процедурой сдвига (A.9-2) в обе стороны. Гаммирование с обратной связью проверено на 48 байтах с заданной синхропосылкой (A.13) в обе стороны.
2. **Круговой прогон.** Простая замена – все длины от 16 до 48 байт, гаммирование с обратной связью – все длины от 1 до 48 байт. В обоих случаях длина шифртекста равна длине открытого текста, а расшифрование возвращает исходный текст побайтово.
3. **Отказы.** Простая замена отклоняет сообщение короче блока и ключ неверной длины; гаммирование с обратной связью отклоняет пустое сообщение, пустой шифртекст и синхропосылку неверной длины.

> **Где нашлась ошибка**
> Самое чувствительное место – перестановка регистров на каждом такте. При её отсутствии шифр работает, но выдаёт неверный шифртекст, а круговые тесты при этом проходят: шифрование и расшифрование одинаково неверны и согласованны между собой. Ошибка вылезла только на примерах

приложения А. Отсюда правило: обязательно сверяться с эталоном, одними круговыми тестами не обойтись.

8. Вопросы на защите

? Что делалось в работе?

Блочный шифр `belt` стандарта СТБ 34.101.31-2011: подстановка H, преобразование G_r, восемь тактов, шифрование и расшифрование блока. Плюс два режима работы: простая замена (п. 6.2) и гаммирование с обратной связью (п. 6.4). Проверка – на примерах приложения А.

? Какие параметры у шифра?

Блок 128 бит, ключ 256 бит, 8 тактов, по 7 тактовых ключей на такт. Подстановка одна: таблица H на 256 байт.

? Чем `belt` отличается от ГОСТ 28147-89?

Блок 128 бит вместо 64, поэтому блок не повторяется в первые 2^{64} сообщений. Это не сеть Фейстеля: за такт меняются все четыре слова, а не половина блока, поэтому хватает 8 тактов вместо 32. Подстановка одна и на 8 бит, а не восемь на 4 бит, и она зафиксирована в стандарте, а не выбирается ведомством. Расшифрование – отдельная процедура с обратным порядком тактовых ключей.

? Что такое сеть Фейстеля и чем она отличается от `belt`?

В сети Фейстеля блок делится на две половины, и каждый такт делает новую половину равной старой, а вторую – старой половине, сложенной с результатом функции F от первой. За счёт этого шифр обратим при **любой**, даже необратимой функции F: при расшифровании половины просто меняются местами, а F применяется повторно. В `belt` половины не меняются местами, все четыре слова преобразуются одновременно, и обратимость обеспечивается тем, что каждая операция внутри такта сама по себе обратима: XOR, сложение и вычитание по модулю 2^{32}, побайтовая подстановка H, циклический сдвиг.

? Почему `belt` не сеть Фейстеля?

В такте преобразуются все четыре регистра, а не половина блока, и операции связаны между собой не только через F. Схема ближе к сети Лэя–Мэсси.

? Зачем нужна раундовая константа?

Это номер такта $\langle i \rangle_{32}$, который добавляется по XOR к промежуточному значению e. Без неё восемь тактов с одинаковой структурой и одинаковыми ключами дали бы эффект слайдовой атаки: зная один такт, можно было бы повторить его на следующем.

? Как в коде сделана перестановка регистров?

Отдельного вызова нет. Первые семь перестановок заданы тем, что следующий такт читает регистры уже в переставленном порядке (`b, d, a, c = beltRound(b, d, a, c, ...)`). Восьмая выполняется тремя обменами в конце функции, потому что следующего такта уже нет.

? Сколько раз за сообщение выполняется перестановка регистров?

Восемь – по одной на такт, перестановка входит в состав такта. Отдельной перестановки при сборке блока нет: формула $Y = b \parallel d \parallel a \parallel c$ просто указывает порядок склеивания регистров.

? Как получаются тактовые ключи?

Ключ делится на восемь 32-битных слов, индекс берётся по модулю восьми: $K_j = \theta_{(j-1) \bmod 8} + 1$. Никаких преобразований над ключом нет, каждое слово просто используется семь раз за такты.

? Почему при расшифровании такты идут в обратном порядке?

Потому что формулы такта не обратимы сами по себе: чтобы отменить такт, нужно начать с последней операции. Стандарт это учитывает – расшифрование описано отдельной процедурой с обратным порядком тактовых ключей и другой перестановкой регистров на конце такта.

? Почему при расшифровании шаги вычисления e не меняются?

Потому что e считается из b и c **до** того, как они изменятся, а последующие шаги разворачивают зависимость: такт в целом обратим, хотя отдельные его шаги – нет.

? Что такое кража шифртекста и зачем она нужна?

Приём, позволяющий зашифровать сообщение, не кратное блоку, **без дополнения**: недостающие до полного блока байты берутся у зашифрованного предпоследнего блока. Выигрыш – длина шифртекста равна длине открытого текста. Плата – сообщение должно быть не короче одного блока, и два последних блока обрабатываются особым образом.

? Почему простая замена не принимает сообщения короче 16 байт?

Кража шифртекста берёт байты у предыдущего блока. Если блок всего один и он неполный, брать неоткуда, и стандарт такой случай не определяет. В программе стоит явная проверка.

? Дополняется ли неполный последний блок в простой замене?

Нет. Вместо дополнения применяется кража шифртекста, поэтому длина шифртекста совпадает с длиной открытого текста.

? Шифруется ли дополненный блок в процедуре сдвига?

Да. Сначала шифруется предпоследний полный блок, затем его хвост склеивается с неполным блоком открытого текста, и **этот** блок шифруется второй раз. Его шифртекст встаёт на место предпоследнего блока, а первые r байт предпоследнего блока – на место неполного.

? Чем гаммирование с обратной связью отличается от простой замены?

Обратная связь. В простой замене блоки шифруются независимо, одинаковые блоки дают одинаковый шифртекст, синхропосылки нет. В гаммировании с обратной связью каждый блок складывается с результатом шифрования предыдущего блока шифртекста, поэтому одинаковые блоки открытого текста дают разные блоки шифртекста. Цена – зашифрование становится строго последовательным.

? Зачем в режиме синхропосылка и почему она не секретная?

Синхропосылка S нужна, чтобы расшифрование с самого начала знало, что положить в регистр обратной связи, – это аналог начального вектора. Секретной она не является: её передача открыто не даёт противнику никаких новых возможностей, зато позволяет не передавать ключ. Брать её нужно случайной: повтор синхропосылки при том же ключе делает два сообщения зависимыми друг от друга.

? Нужна ли обратная функция F^{-1} в режиме гаммирования?

Нет. Шифр вырабатывает блок, который складывается с текстом по XOR, и этот блок одинаков для обоих направлений – нужен только F. Это экономит половину схемы и в железе, и в программе.

? Почему в режиме гаммирования не нужно дополнение?

Потому что последний блок просто короче, и от результата шифрования берутся только первые $|X_n|$ бит – ровно столько, сколько нужно. Ничего дописывать не надо, длина сохраняется автоматически.

? Что такое самосинхронизация и есть ли она здесь?

Состояние режима – последний принятый блок шифртекста. Если блок потерян, искажён или вставлен лишний, то через один блок регистр снова содержит верные данные и расшифрование продолжается правильно; безвозвратно теряется только один блок. Это свойство есть у режима гаммирования с обратной связью и отсутствует у простой замены.

? Как ошибка в шифртексте распространяется в каждом режиме?

Простая замена: портится весь соответствующий блок открытого текста. Гаммирование с обратной связью: портится один бит своего блока и весь следующий блок, дальше поток восстанавливается.

? Почему в тестах нужно сверяться с примером из стандарта, раз есть круговые проверки?

Потому что круговая проверка ловит только ошибку, согласованную в обе стороны. В нашем случае именно так и было: без перестановки регистров шифрование и расшифрование ошибались согласованно, и круговые тесты проходили, а пример из приложения А показывал несовпадение.

? Как проверялась корректность реализации?

На примерах приложения А: шифрование и расшифрование блока, простая замена на 48 и на 47 байтах в обе стороны, гаммирование с обратной связью в обе стороны. Плюс круговые прогоны на всех длинах и проверки отказов на коротких входах.

? Что ещё есть в стандарте, чего нет в работе?

Сцепление блоков (п. 6.3), счётчик (п. 6.5), имитовставка (п. 6.6), совместное шифрование и имитозащита (п. 6.7, 6.8), хеширование (п. 6.9), а также расширение ключей 128 и 192 бит (п. 7). Задание ограничено двумя режимами шифрования.

? Что делает шифрование, а что – имитовставка?

Шифрование даёт конфиденциальность: без ключа прочитать нельзя. Имитовставка даёт целостность и подлинность: она вычисляется на ключе, поэтому получатель может проверить, что сообщение не меняли и что отправитель знает ключ. Шифрования целостности не даёт – в режиме гаммирования с обратной связью противник может инвертировать выбранные биты шифртекста и предсказуемо изменить открытый текст, не зная ключа.

? Чем в вашей работе обмениваются при передаче?

Ключ передаётся отдельно, вне шифртекста. В режиме гаммирования с обратной связью синхропосылка передаётся открыто впереди шифртекста и каждый раз новая. В простой замене ничего, кроме ключа, передавать не нужно.

? Считается ли реализация безопасной для применения?

Как учебная – да: корректность подтверждена эталонными примерами. Для применения к реальным данным режимы в таком виде непригодны, потому что не обеспечивают целостность и подлинность; в самом стандарте для этого есть отдельный режим совместного шифрования и имитозащиты.